The EAOS methodology defines a structured lifecycle for
Designing, deploying, and governing enterprise-grade AI systems

Project: Standard Framework Project

0.1.1 AI Agent SaaS Framework

Phase: 0. SaaS Foundation

[산출물 ID: 0.1.1 AI Agent SaaS Framework.docx]

① 이 문서 (Framework) : 무엇을, 왜 이렇게 결정했는가?
② Implementation Guide : 어떻게 코드로 구현하는가?
③ Quick Reference Card : 개발 중 규칙 즉시 참조

문서 번호	0.1.1 AI Agent SaaS Framework.docx 설계지도(Why & What)
버전	v1.0
작성일	2026
작성자	KY Song
분류	CONFIDENTIAL
적용 대상	Standard Framework 프로젝트 전체

Table of Contents

전체 설계 영역 지도 3
1. Platform Foundation 4
결정 1 — 기술 스택 4
결정 2 — 환경 전략 4
결정 3 — Feature Flag 4
2. Identity & Access 5
결정 4 — 인증 방식 5
결정 5 — 인가 모델 (RBAC) 5
결정 6 — 소셜 로그인 우선순위 5
3. Tenant Management 6
결정 7 — 멀티테넌시 격리 전략 6
결정 8 — 도메인 계층 구조 6
결정 9 — 구독 플랜 6
4. Data Governance 7
결정 10 — 데이터 등급 분류 7
결정 11 — 보존 정책 · 결정 12 — Soft Delete 7
5. Application Architecture 8
결정 13 — 서비스 아키텍처 8
결정 14 — 레이어드 아키텍처 · 결정 15 — 이벤트 처리 8
6. Integration & API 9
결정 16 — API 설계 원칙 9
결정 17 — 표준 응답 포맷 9
7. Observability & Operations 10
결정 18 — 구조화 로그 10
결정 19 — 모니터링 임계값 10
8. Governance & Standards 11
결정 20 — CI/CD · 결정 21 — 배포 11
결정 22 — 보안 정책 11
9. Localization & Globalization 12
결정 23 — 다국어 · 결정 24 — 시간대 · 결정 25 — 금액 12
10. AI Agent Architecture 13
결정 26 — Agent 실행 프레임워크 13
결정 27 — Agent 유형 · 결정 28 — Memory 전략 13
결정 29 — AI 안전성 (3중 Guardrail) 13
11. Architecture Decision Record (ADR) 14
12. 승인 (Approval) 15

전체 설계 영역 지도

Layer	설계 영역	이 문서의 결정 번호
01	Platform Foundation — 기반 구조, 기술 스택, 환경	결정 1, 2, 3
02	Identity & Access — 인증 · 인가 · 세션 · OAuth	결정 4, 5, 6
03	Tenant Management — 멀티테넌시, 조직, 구독	결정 7, 8, 9
04	Data Governance — 데이터 보안 · 보존 · 감사	결정 10, 11, 12
05	Application Architecture — 서비스 구조, 모듈	결정 13, 14, 15
06	Integration & API — API 설계, Webhook	결정 16, 17
07	Observability — 로깅, 메트릭, 알림	결정 18, 19
08	Governance & Standards — 컨벤션, 보안, CI/CD	결정 20, 21, 22
09	Localization — 다국어, 시간대, 통화	결정 23, 24, 25
10	AI Agent Architecture — Agent 설계 · 메모리 · 안전	결정 26, 27, 28, 29

1. Platform Foundation

결정 1 — 기술 스택

각 기술은 ´팀 숙련도 + AI 생태계 호환성´을 최우선 기준으로 선택했다.

계층	선택	탈락한 대안	결정 이유
Backend	FastAPI	Django REST, Express	async 완전 지원 + 자동 OpenAPI + Python AI 생태계
ORM	SQLAlchemy 2.0 async	Tortoise, Prisma	가장 성숙한 Python async ORM + Alembic 통합
Frontend	Next.js 14	Vite+React, Remix	SSR/SSG/ISR 혼합 + TypeScript 기본 + SEO
Primary DB	PostgreSQL 16	MySQL, MongoDB	ACID + JSONB + pgvector 단일 DB
Cache/Queue	Redis 7	Memcached, RabbitMQ	In-Memory + Pub/Sub + Session 단일 도구
AI Orchestration	LangChain + LangGraph	AutoGen, CrewAI	상태 기반 워크플로우 표준, 생태계 최대
Vector DB	pgvector → Qdrant	Pinecone, Weaviate	초기 PG 통합, 규모 증가 시 전문 DB 전환
UI Component	shadcn/ui + Tailwind	MUI, Ant Design	컴포넌트 소유권 + 번들 최소화 + 커스터마이징 자유

결정 2 — 환경 전략

환경	목적	인프라	왜 이 환경이 필요한가?
local	개발자 개인 개발	Docker Compose	—
dev	팀 통합 테스트	Railway/Render	main 브랜치가 곧 staging 이 되는 혼란 방지
staging	프로덕션 검증	AWS (운영 동일)	프로덕션 배포 전 마지막 검증 관문
production	실제 서비스	AWS ECS + RDS	—

결정 3 — Feature Flag

왜 필요한가? 완성되지 않은 기능이 코드에 있어도 사용자에게 노출되지 않게 하고, 플랜별 기능을 선별 제공한다.

플래그 키	기본값	활성화 조건
ai_agent_enabled	false	pro 이상
ai_agent_advanced	false	enterprise
api_access	false	pro 이상
sso_saml	false	enterprise
export_excel	false	pro 이상

2. Identity & Access

결정 4 — 인증 방식

방식	고려 이유	탈락 이유	채택
JWT ✓	Stateless → 수평 확장 용이	토큰 강제 만료 어려움 → Rotate 로 보완	✓
Session	즉시 무효화 가능	Redis 의존 + 수평 확장 복잡	✕
Passwordless	보안 강화	UX 변화 크고 이메일 의존	✕

- Access Token : 만료 15 분 | 저장: 클라이언트 메모리 (Zustand) ← XSS 방어
- Refresh Token : 만료 30 일 | 저장: HttpOnly Cookie (SameSite=Strict) ← CSRF 방어
- Refresh Rotate: 사용할 때마다 새 토큰 발급 + 이전 토큰 즉시 Blacklist ← 탈취 감지

결정 5 — 인가 모델 (RBAC)

왜 RBAC 인가? ABAC 은 표현력이 높지만 초기 B2B SaaS 는 RBAC 으로 95%를 충족한다. 이후 ABAC 을 레이어로 추가 가능.

퍼미션	owner	admin	member	viewer
org:delete — 조직 삭제	✓	✕	✕	✕
org:settings — 설정 변경	✓	✓	✕	✕
members:invite — 멤버 초대	✓	✓	✕	✕
data:create — 데이터 생성	✓	✓	✓	✕
data:read — 데이터 조회	✓	✓	✓	✓
data:delete — 데이터 삭제	✓	✓	✕	✕
agent:run — Agent 실행	✓	✓	✓	✕
billing:manage — 결제 관리	✓	✓	✕	✕

결정 6 — 소셜 로그인 우선순위

Provider	우선순위	이유
Google OAuth2	1 순위 (필수)	B2B 고객 대부분 Google Workspace 사용. 프로필 자동 수집
GitHub OAuth	2 순위	개발자 향 제품. 내부 테스트에도 유용
Microsoft Entra	3 순위 (Enterprise)	대기업 고객 SAML/SSO 확장 시 필요

3. Tenant Management

결정 7 — 멀티테넌시 격리 전략

전략	격리 수준	운영 비용	채택 여부	이유
Silo (DB 분리)	최상	테넌트 수 × 비용	✕	초기 스타트업에 부적합
Bridge (Schema 분리)	높음	관리 복잡도 증가	✕	마이그레이션 관리 어려움
Pool (Row 분리) ✓	중간	낮음	✓	초기 비용 최소화, Bridge 전환 경로 확보

> Pool 의 단점 보완책: SQLAlchemy Session Event 로 WHERE organization_id = :tenant_id 자동 주입 → 개발자 실수 방지

결정 8 — 도메인 계층 구조

```
Organization (테넌트 = 회사)          최상위 계약 단위
  └── Workspace (프로젝트/팀 공간)    나중에 추가하면 DB 마이그레이션 비용 큼 → 처음부터
        └── Member (접근 사용자)
              └── User (실제 사람)
Subscription (결제/플랜)              Organization 단위
```

결정 9 — 구독 플랜

항목	Free	Starter	Pro	Enterprise
가격	무료	$29/월	$99/월	협의
멤버 수	3 명	10 명	50 명	무제한
Agent 실행/월	10 회	100 회	1,000 회	무제한
API 접근	✕	✕	✓	✓
SSO/SAML	✕	✕	✕	✓

4. Data Governance

결정 10 — 데이터 등급 분류

등급	예시	처리 방법
Public	공개 문서, 마케팅	일반 저장
Internal	업무 데이터, 프로젝트	테넌트 격리
PII	이메일, 전화번호	마스킹 + 암호화
Restricted	API 키, 비밀번호	AES-256 + AWS KMS

결정 11 — 보존 정책 · 결정 12 — Soft Delete

데이터	보존 기간	근거
감사 로그	3 년	SOC2, ISO27001
결제 기록	5 년	세법
사용자 계정	탈퇴 후 30 일 → Hard Delete	개인정보보호법
AI 대화 기록	1 년 (테넌트 설정 가능)	내부 정책

> Soft Delete 이유: Hard Delete 는 복구 불가. 법적 분쟁 · 감사 시 데이터가 반드시 필요한 경우가 생긴다.

5. Application Architecture

결정 13 — 서비스 아키텍처

아키텍처	장점	단점	채택 시점
Monolith	개발 빠름, 디버깅 쉬움	팀 규모 커지면 병목	팀 10 명 이하
Modular Monolith ✅	빠른 개발 + 분리 경로	모듈 경계 규율 필요	팀 5~20 명 ✅
Microservice	독립 배포, 기술 다양성	운영 복잡도 폭발	팀 20 명+, 도메인 안정화 후

> Modular Monolith 규칙: 도메인 간 직접 import 금지. 서비스 레이어 또는 내부 이벤트를 통해서만 통신.

결정 14 — 레이어드 아키텍처 · 결정 15 — 이벤트 처리

레이어	책임	금지 사항
Router	HTTP 요청/응답	비즈니스 로직 직접 구현
Service	비즈니스 로직 전담	DB 직접 접근
Repository	DB 쿼리 전담	비즈니스 로직 포함
Model	DB 스키마 정의	로직 포함

> 이벤트 처리 단계적 전략:
> 현재: 내부 이벤트 버스 (메모리, 동일 프로세스) — 인터페이스 동일 유지
> 이후: Kafka / AWS SQS 전환 시 비즈니스 로직 변경 없음

6. Integration & API

결정 16 — API 설계 원칙

원칙	설명	예시
복수 명사 리소스	동사 URL 금지	/agents /organizations
HTTP 메서드로 행위	URL 에 동사 없음	POST /agents (생성), DELETE /agents/{id}
계층 구조 중첩	최대 2 단계	/organizations/{id}/members
버전 접두사	URL 에 명시	/api/v1/
필터는 쿼리 파라미터	경로에 포함 금지	?status=active&sort=-created_at

> 버전 정책: Breaking Change(필드 제거, 타입 변경) 시에만 v2. 구버전 6 개월 유지 후 폐기.

결정 17 — 표준 응답 포맷

```
// 성공 (단건)
{ "success": true, "data": {...}, "meta": { "request_id": "..." } }

// 성공 (목록)
{ "success": true, "data": { "items": [...], "total": 100, "page": 1, "size": 20 } }

// 실패
{ "success": false, "error": { "code": "ERR_AUTH_002", "message": "..." } }
```

> 이유: Frontend 응답 처리를 단일화. 에러 코드 기반 다국어 메시지 매핑 가능.

7. Observability & Operations

결정 18 — 구조화 로그

왜 JSON 로그? 텍스트 로그는 검색·집계 어려움. JSON 이면 CloudWatch Insights / Grafana Loki 에서 즉시 쿼리 가능.

항목	내용
모든 로그에 포함	timestamp · level · trace_id · org_id · user_id · request_id
로그에서 반드시 제외	password · token · api_key · secret · card_number
라이브러리	structlog (Python) — JSON 자동 직렬화

결정 19 — 모니터링 임계값

메트릭	경고	즉시 대응	채널
API 에러율	> 1%	> 5%	Slack → PagerDuty
API P99 응답시간	> 2 초	> 5 초	Slack → PagerDuty
Agent 실행 성공률	< 95%	—	Slack #alerts
DB 연결 풀 사용률	> 80%	—	Slack #alerts

8. Governance & Standards

결정 20 — CI/CD · 결정 21 — 배포

전략	적용 환경	이유
Rolling Update	Dev	빠른 배포, 약간의 다운타임 허용
Blue-Green	Production	무중단 + 즉시 롤백
Canary	대규모 기능 출시	5% → 25% → 100% 점진적 이동
Feature Flag	기능 단위	코드 배포 없이 on/off

결정 22 — 보안 정책

영역	결정	이유
전송 암호화	TLS 1.3 강제	중간자 공격 방어
저장 암호화	AES-256 + AWS KMS	유출 시 복호화 불가
비밀번호 해시	bcrypt cost=12	무차별 대입 공격 방어
Rate Limiting	IP 100 req/min · API Token 1,000 req/min	DoS 방어
로그인 실패	5 회 / 15 분 잠금	Brute-force 방어

9. Localization & Globalization

결정 23 — 다국어 · 결정 24 — 시간대 · 결정 25 — 금액

결정	내용	이유
다국어 지원	ko(기본) → en → ja. next-intl / Babel	나중에 추가하면 모든 UI 문자열 일괄 교체 필요
시간대 저장	DB = UTC, API 응답 = ISO 8601 UTC	다국가 서비스 시간대 변환 버그 원천 차단
금액 저장	정수 최소 단위: $10.00 → 1000 (cents)	부동소수점 반올림 오차로 결제 금액 오류 방지

10. AI Agent Architecture

결정 26 — Agent 실행 프레임워크

프레임워크	특징	탈락 이유	채택
LangGraph ✅	상태 기반 그래프, 조건부 분기	—	✅
LangChain Agents	단순 ReAct Loop	복잡한 멀티에이전트 흐름 표현 어려움	❌
AutoGen	대화형 멀티에이전트	워크플로우 제어 어려움	❌
CrewAI	역할 기반 에이전트 팀	커스터마이징 제약	❌

결정 27 — Agent 유형 · 결정 28 — Memory 전략

메모리 유형	저장 위치	TTL	이유
단기 (대화 컨텍스트)	Redis	세션 종료	빠른 읽기/쓰기
중기 (대화 요약)	PostgreSQL	90 일	영속적 보존 필요
장기 (지식 베이스)	pgvector	영구	벡터 유사 검색 필요
작업 (실행 중 상태)	Redis	실행 종료	임시 데이터

> pgvector 를 먼저 쓰는 이유: 초기엔 PG 통합으로 인프라 단순화. 임베딩 수천만 건 초과 시 Qdrant 전환.

결정 29 — AI 안전성 (3 중 Guardrail)

> [입력 단계] 최대 토큰 제한 · OpenAI Moderation · 프롬프트 인젝션 감지 · PII 마스킹
> [실행 단계] max_iterations 제한 · 타임아웃 120 초 · 허용 Tool 만 · URL 화이트리스트
> [출력 단계] 유해 콘텐츠 재검증 · PII 마스킹 · 환각 신뢰도 체크

11. Architecture Decision Record (ADR)

설계를 변경할 때마다 이 목록에 추가하고 이유와 재검토 조건을 기록한다.

#	결정	상태	이유 요약	재검토 조건
001	Backend: FastAPI	✅	async + AI 생태계	팀 기술 스택 전환 시
002	DB: PostgreSQL + pgvector	✅	ACID + 벡터 단일 DB	임베딩 수천만 건 초과 시
003	인증: JWT Rotate	✅	Stateless + 탈취 감지	—
004	멀티테넌시: Pool	✅	초기 비용 최소화	테넌트 10,000 개 초과 시
005	Modular Monolith	✅	개발 속도 + 분리 경로	팀 20 명 초과 시
006	LangChain + LangGraph	✅	상태 기반 워크플로우	더 나은 프레임워크 등장 시
007	pgvector → Qdrant	📋	초기 단순화	임베딩 규모 확대 시
008	내부 이벤트 → Kafka	📋	초기 단순화	서비스 분리 시
009	React Query + Zustand	✅	서버/클라이언트 상태 분리	—
010	Docker → ECS → K8s	📋	단계별 확장	월 트래픽 1M 초과 시

12. 승인 (Approval)

구분	내용	서명	날짜
작성자			
검토자 (AI Architect)			
승인자 (Business Owner)			
버전	v1.0		